

SYDE572 – Assignment 1

PART ONE. Find shortest Distance from a Point to a Function

Finding the shortest distance from a specific point (x_0, y_0) to a continuous mathematical function $y = f(x)$ comes down to an optimization problem.

Depending on your use case—whether you are writing code for a computer graphics engine or solving a calculus homework problem—there are two primary ways to approach this: analytically (using exact calculus) or numerically (using iterative algorithms).

1. The Mathematical Foundation (The Objective Function)

1. The Euclidean distance d between a given point (x_0, y_0) and any arbitrary point (x, y) on the function's curve is defined as:

$d = \sqrt{(x - x_0)^2 + (y - y_0)^2}$ Because a square root is computationally expensive and monotonically increasing, algorithms always optimize the squared distance (D) instead. The point x that minimizes D will also minimize d : $D(x) = (x - x_0)^2 + (f(x) - y_0)^2$

2. The Analytical Approach (Calculus)

1. If you have a differentiable, smooth function (like a polynomial or trigonometric function), you can solve for the exact distance using calculus.
 1. Step 1: Take the derivative of the squared distance function with respect to x : $D'(x) = 2(x - x_0) + 2(f(x) - y_0)f'(x)$
 2. Step 2: Set the derivative equal to zero to find the critical points: $0 = (x - x_0) + (f(x) - y_0)f'(x)$
 3. Step 3: Solve for x . This equation fundamentally states that the vector from the point to the curve must be perpendicular (orthogonal) to the tangent of the curve at that point.
 4. Step 4: Plug the resulting x back into the original distance formula to find the minimum distance.

1. The Numerical Approach (Algorithms for Code)

1. In programming, solving $D'(x) = 0$ symbolically can be impossible if $f(x)$ is highly complex. Instead, we use numerical root-finding or optimization algorithms.

Algorithm A: Newton-Raphson Method (Fastest if differentiable)

If you can easily calculate the first and second derivatives of $f(x)$, you can use Newton's method to find where $D'(x) =$

0.

```
def find_distance_newton(x0, y0, f, df, ddf, initial_guess=0.0,
    tolerance=1e-7, max_iter=100):
    x = initial_guess
    for _ in range(max_iter):
        # D'(x)
        D_prime = 2 * (x - x0) + 2 * (f(x) - y0) * df(x)
        # D''(x)
        D_double_prime = 2 + 2 * (df(x)**2) + 2 * (f(x) - y0) * ddf(x)
        # Newton-Raphson update step
        next_x = x - D_prime / D_double_prime
        if abs(next_x - x) < tolerance:
            break
    x = next_x
    shortest_distance = ((x - x0)**2 + (f(x) - y0)**2)**0.5
    return shortest_distance, x
```

Algorithm B: Golden Section Search (Best if derivatives are unknown)

If you don't know the derivative of $f(x)$ but you can restrict your search to a specific interval $[a, b]$, a Golden Section

Search or Bisection Method is incredibly robust.

```
import math
def golden_section_search(x0, y0, f, a, b, tolerance=1e-7):
    phi = (1 + math.sqrt(5)) / 2
    resphi = 2 - phi
    x1 = a + resphi * (b - a)
    x2 = b - resphi * (b - a)
    def dist_sq(x): return (x - x0)**2 + (f(x) - y0)**2
    f_x1 = dist_sq(x1)
    f_x2 = dist_sq(x2)
    while abs(b - a) > tolerance:
        if f_x1 < f_x2:
            b = x2
            x2 = x1
            f_x2 = f_x1
            x1 = a + resphi * (b - a)
            f_x1 = dist_sq(x1)
        else:
            a = x1
            x1 = x2
            f_x1 = f_x2
            x2 = b - resphi * (b - a)
```

```
f_x2 = dist_sq(x2)
best_x = (a + b) / 2
return math.sqrt(dist_sq(best_x)), best_x
```

Solve using both the analytical approach and using the 2 numerical approaches by hand for the following function $y = x^2 + 5$ for the following points (0,0),(-4,0),(-8,0),(2,0),(6,0).

For the 2 numerical solutions, code them as a python code to compute the distance from any point to any parabola function. Demonstrate that your routine works by applying the above parabola and data points and also provide some results for other functions and points.

Modify your code slightly to show that it also works for non-polynomial functions (e.g., exponential, logarithmic, variables in denominator, or radical root functions).

Provide plots showing intermediate steps of the numerical methods with the points and equations labelled.

Deliverable for part 1: you must submit your project webpage. **Make beautiful plots. Make your webpage visually appealing. Aesthetics matter!**

1. **Host your webpage online.** We recommend using GitHub Pages but any hosting service is acceptable. You may organize your project in any directory structure you prefer.
2. **Submit a PDF version to learn.**

Grading Rubric

This assignment will be graded out of **100** points, as follows:

Distance to a function (50 points total), Fitting a function (50 points total)

For analytical results:

- **+50%:** correct
- **+30%:** Some defects
- **0%:** No effort

For numerical:

- **+50%:** Thorough explanation / approach / good presentation, works for other cases, same as analytical.
- **+30%:** Explanation present, could go further in depth.
- **+20%:** Minimal explanation on webpage.
- **0%:** No section / no explanation on webpage.

PART TWO. fitting to an equation

If we have the following set of points $(0,0.5), (2,3.5), (1,1.5), (3,7.5)$, we want to fit a parabola and a line to these points by minimizing the min squared error of the fitted function. Set up the MSE function. You will have at least 2 parameters to optimize for, the slope and intercept and similarly for the parabola. Using an analytical method and a numerical method (Newton-Raphson multivariate), firstly optimize for one parameter assuming a particular value for the other parameter and then optimize for the 2nd parameter in each iteration step. What is the MSE you get for the line and for the parabola? Show the worked out solution by hand and also supply python code (your own code) to automate the procedure. Produce plots of intermediate steps.

Using GITHUB

TL;DR: If you make a repo called `<your_github_username>.github.io`, GitHub will host your site for free! Then it's as simple as adding an `index.html` file to the repo and building on top of it.

1. **Log in to GitHub:**
2. **Create a New Repository:**
 - [Go here](#), or to your GitHub dashboard and click the green "New" button to create a new repository.
 - Name your repository `<your_github_username>.github.io` (replace `<your_github_username>` with your actual GitHub username).

- Set the repository to be **public** -- course staff will need access when grading.
- Start with a README file so the repo isn't empty.
- Click the "Create repository" button.

Step 2: Add an index.html File

1. **Navigate to Your New Repository:** After creating the repository, you'll be redirected to the repository page.

2. **Create an index.html File:**

- Click the "Add file" button, then select "Create new file."
- Name the file index.html.
- In the file editor, you can start with something simple like:
- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<title>My Portfolio</title>`
- `<style>`
- `body {`
- `font-family: Arial, sans-serif;`
- `margin: 20px;`
- `}`
- `</style>`
- `</head>`
- `<body>`
- `<h1>Welcome to My SYDE671 Portfolio!</h1>`
- `<a href="/1/index.html">Assignment 1</a>`
- `</body>`

- `</html>`
- Scroll down and commit your new file by clicking "Commit new file."

Step 3: Access Your Website

1. **Wait a Few Minutes:** GitHub will automatically deploy your site. They say this may take up to 10 minutes. This is your time to brew some tea.
2. **Visit Your New Site:**
 - Go to `https://<your_github_username>.github.io` in your web browser.
 - You should see the content of your `index.html` file displayed as your website.

Step 4: Assignment Pages

Right now, if you copied the example HTML above, you'll notice the 'Project 1' URL returns 404 as it links to a page that doesn't (yet) exist.

For each project, you should create a separate directory/subpage that you'll link to from the main page. An example directory structure is below:

```
<your_github_username>.github.io/
├──index.html  (main page)
├──1/
│   ├──index.html
│   └──media/
│       ├──img1.jpg
│       ├──:
│       └──imgN.jpg
└──2/
    ├──:
```

Each time you commit changes, GitHub Pages will update your site in up to 10 minutes. For this reason, we recommend cloning and editing locally so you can witness your modifications real-time and then push once satisfied.

Step 5: Customization

- You can customize the look of your site by modifying the CSS `<style>` tag in the `<head>` of your `index.html` file.

- Alternatively, you could create a separate style.css file; if you wanted to re-use this styling across projects pages, you'd place it in the root directory and import it *absolutely* with `<link rel="stylesheet" href="style.css">` in the `<head>` tag.
- Otherwise, you could place the style file in the project directory and reference the *relative* path as `<link rel="stylesheet" href="./style.css">`.
 - This is similar to images: ``

We aren't going to be sticklers about the design of your site, but it should be easy to navigate and understand. You shouldn't bog the site with deliverable images that take up the whole page, for example.